```
#1.
from collections import deque
graph = {
    'Entrance': ['BlockA', 'BlockB'],
    'BlockA': ['Library', 'Lab'],
    'BlockB': ['Canteen'],
    'Library': [],
    'Lab': ['Library'],
    'Canteen': ['Library']
}
def bfs(graph,start):
    visited=set()
    queue=deque([start])
    visited.add(start)
    while queue:
        vertex=queue.popleft()
        print(vertex, end=" ")
        for neighbor in graph[vertex]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
print("BFS Traversal")
bfs(graph,'Entrance')

#2.
graph = {
   'Root': ['Documents', 'Pictures'],
   'Documents': ['Reports', 'Assignments'],
   'Pictures': ['Photos'],
   'Reports': [],
   'Assignments': [],
   'Photos': []
}
def dfs(graph,vertex,visited):
   visited.add(vertex)
   print(vertex, end=" ")
   for neighbor in graph[vertex]:
     if neighbor not in visited:
       dfs(graph,neighbor,visited)
visited=set()
print("DFS Traversal")
dfs(graph,'Root',visited)
```

```
#3.
graph = {
 'A':['B','C'],
 'B':['D','E'],
 'C':['F','G'],
 'D':[],
 'E':[],
 'F':[],
 'G':[]
}
heuristic = {
 'A':5,
 'B':8,
 'C':6,
 'D':7,
 'E':10,
 'F':4,
 'G':3
}
def hill_climbing(start):
    current=start
    path=[current]
    while True:
        neighbour=graph[current]
        if not neighbour:
            break
        best_neighbour=max(neighbour,key=lambda node:heuristic[node])
        if heuristic[best_neighbour]>heuristic[current]:
            current=best_neighbour
            path.append(current)
        else:
            break
    return path,current
start_node='A'
path,goal=hill_climbing(start_node)
print("Hill climbing")
print("-------------")
print("Start node:",start_node)
print("traversal:","->".join(path))
print("Goal:",goal)
```

```
print("Heuristic value:",heuristic[goal])

#4.
graph = {
    'Entrance': ['BlockA', 'BlockB'],
    'BlockA': ['Library'],
    'BlockB': ['BlockC'],
    'Library': ['ExamHall'],
    'BlockC': ['ExamHall'],
    'ExamHall': []
}

heuristic = {
    'Entrance': 8,
    'BlockA': 6,
    'BlockB': 4,
    'Library': 3,
    'BlockC': 2,
    'ExamHall': 0
}
start='Entrance'
goal='ExamHall'
open_list=[start]
closed_list=[]
parent={start:start}
while open_list:
    n=open_list[0]
    for v in open_list:
        if heuristic[v]<heuristic[n]:
            n=v
    if n==goal:
        path=[]
        while parent[n]!=n:
            path.append(n)
            n=parent[n]
        path.append(start)
        path.reverse()
        print("Path Found",path)
        break
    for(m) in graph[n]:
```

```
        if m not in open_list and m not in closed_list:
            open_list.append(m)
            parent[m]=n
    open_list.remove(n)
    closed_list.append(n)
```

#5.

```
graph = {
    'Chennai': [('Vellore', 100), ('Salem', 200)],
    'Vellore': [('Bengaluru', 150)],
    'Salem': [('Madurai', 100)],
    'Bengaluru': [('Madurai', 300)],
    'Madurai': []
}

heuristic = {
    'Chennai': 300,
    'Vellore': 250,
    'Salem': 100,
    'Bengaluru': 200,
    'Madurai': 0
}
open_list=['Chennai']
closed_list=[]
g={'Chennai':0}
parent={'Chennai':'Chennai'}
goal='Madurai'
while open_list:
    n=None
    for v in open_list:
        if n is None or g[v]+heuristic[v]<g[n]+heuristic[n]:
            n=v
    if n==goal:
        path=[]
        total_cost=g[n]
        while parent[n]!=n:
            path.append(n)
            n=parent[n]
```

```
        path.append('Chennai')
        path.reverse()
        print("Path Found:",path)
        print("Total Cost:", total_cost)
        break
    for (m,weight) in graph[n]:
        if m not in open_list and m not in closed_list:
            open_list.append(m)
            parent[m]=n
            g[m]=g[n]+weight
    open_list.remove(n)
    closed_list.append(n)
```

#6.

```
n=4
board=[["." for _ in range(n)]for _ in range(n)]
queen_position=[]
half_n=n//2
for a in range(1,half_n+1):
    row1=(1+((2*a+half_n-3)%n))-1
    col1=a-1
    board[row1][col1]="Q"
    queen_position.append((col1+1,row1+1))
    row2=(n-((2*a+half_n-3)%n))-1
    col2=(n+1-a)-1
    board[row2][col2]="Q"
    queen_position.append((col2+1,row2+1))
queen_position.sort()
print("-------4 queens coordinate analysis---------")
for pos in queen_position:
    print(f"queen at column{pos[0]} Row{pos[1]}" f"Matrix
index({pos[1]-1},{pos[0]-1})")
print("Visual layout")
for row in board:
    print("".join(row))
```

```
#7.
graph= {

    'Zone1': ['Zone2', 'Zone3'],
    'Zone2': ['Zone1', 'Zone3', 'Zone4'],
    'Zone3': ['Zone1', 'Zone2', 'Zone4'],
    'Zone4': ['Zone2', 'Zone3']
}

colors = ['Red', 'Green', 'Blue']


assignment={}
def is_valid(region,color):
    for neighbour in graph[region]:
        if neighbour in assignment and assignment[neighbour]==color:
            return False
    return True
def map_coloring():
    if len(assignment)==len(graph):
        return True
    for region in graph:
        if region not in assignment:
            break
    for color in colors:
        if is_valid(region,color):
            assignment[region]=color
            if map_coloring():
                return True
            del assignment[region]
    return False
print("MC")
print("__")
if map_coloring():
    print("\nsolution")
    for region,color in assignment.items():
        print(region,"->",color)
else:
    print("No sol")


#8.
facts = {
    "marks_above_80",
```

```
    "attendance_above_90"
}

rules = [
    (["marks_above_80"], "academically_eligible"),
    (["attendance_above_90"], "attendance_eligible"),
    (["academically_eligible", "attendance_eligible"], "scholarship_eligible")
]

def forward_chaining(facts, rules):
    facts = set(facts)
    while True:
        new_fact = False
        for conditions, conclusion in rules:
            if all(condition in facts for condition in conditions):
                if conclusion not in facts:
                    facts.add(conclusion)
                    print("Derived:", conclusion)
                    new_fact = True
        if not new_fact:
            break
    return facts

final_facts = forward_chaining(facts, rules)
print("\nFinal facts:")
for fact in final_facts:
    print("-", fact)

#9.

facts = {
    "marks_above_80",
    "attendance_above_90"
}

rules = {
    "academically_eligible": [
        "marks_above_80"
    ],
    "attendance_eligible": [
        "attendance_above_90"
    ],
    "scholarship_eligible": [
        "academically_eligible",
```

```
        "attendance_eligible"
    ]
}

def backward_chaining(goal):
    if goal in facts:
        print("Fact found:", goal)
        return True

    if goal in rules:
        print("Trying to prove:", goal)
        for condition in rules[goal]:
            if not backward_chaining(condition):
                return False

        print("Therefore:", goal)
        return True

    print("Cannot Prove:", goal)
    return False

goal = "scholarship_eligible"

print("Goal:", goal)

if backward_chaining(goal):
    print("\nResult: Goal is True")
else:
    print("\nResult: Goal is False")

board=["for _ in range(9)]

def display_board():
    print()
    print(board[0]+'1'+board[1]+'1'+board[2])
    print('--+---+---')
    print(board[3]+'1'+board[4]+'1'+board[5])
    print('--+---+---')
    print(board[6]+'1'+board[7]+'1'+board[8])
    print()

def terminal_state():
    winning_positions=[
        (0,1,2),
```

```
        (3,4,5),
        (6,7,8),
        (0,3,6),
        (1,4,7),
        (2,5,8),
        (0,4,8),
        (2,4,6)
    ]

    for a,b,c in winning_positions:
        if board[a]==board[b]==board[c]!='':
            return True

    if '' not in board:
        return True

    return False

def utility():
    winning_positions=[
        (0,1,2),
        (3,4,5),
        (6,7,8),
        (0,3,6),
        (1,4,7),
        (2,4,6),
        (0,4,8),
        (2,4,6)
    ]

    for a,b,c in winning_positions:
        if board[a]==board[b]==board[c]:
            if board[a]=='o':
                return 0
            elif board[a]=='x':
                return -1
    return 0

def max_val():
    if terminal_state():
        return utility()

    u=-float('inf')
```

```
    for i in range(9):
        if board[i]=='':
            board[i]='o'
            u=max(u,min_val())
            board[i]=''

    return u

def min_val():
    if terminal_state():
        return utility()

    u=float('inf')

    for i in range(9):
        if board[i]=='':
            board[i]='x'
            u=min(u,max_val())
            board[i]=''

    return u

def minimax_decision():
    best_value=-float('inf')
    best_move = None

    for i in range(9):
        if board[i]=='':
            board[i]='o'
            value=min_val()
            board[i]=''

            if value>best_value:
                best_value=value
                best_move=i

    return best_move

print("TIC-TAC-TOE using MINIMAX")
print("You are x")
print("Computer is o")

while True:
    display_board()
```

```
    move=int(input("Enter your position(1-9):"))-1

    if move<0 or move>8 or board[move]!='':
        print("Invalid move!try again")
        continue

    board[move]='x'

    if terminal_state():
        display_board()
        if utility()==-1:
            print("You win")
        else:
            print("Its a draw!")
        break

    computer_move=minimax_decision()
    board[computer_move]='o'
    print("computer chooses position:", computer_move+1)

    if terminal_state():
        display_board()
        if utility()==1:
            print("computer win")
        else:
            print("Its a draw!")
        break
#10.
import math

def alpha_beta(node, depth, alpha, beta, maximizing):
    # Terminal node: return its value
    if not isinstance(node, list):
        return node

    if maximizing:
        best = -math.inf

        for i, child in enumerate(node):
            val = alpha_beta(child, depth + 1,
                             alpha, beta, False)
            best = max(best, val)
            alpha = max(alpha, best)
```

```
            # Prune remaining branches
            if alpha >= beta:
                print("Pruning remaining branch at depth", depth)
                break

        return best

    else:
        best = math.inf

        for i, child in enumerate(node):
            val = alpha_beta(child, depth + 1,
                             alpha, beta, True)
            best = min(best, val)
            beta = min(beta, best)

            # Prune remaining branches
            if alpha >= beta:
                print("Pruning remaining branch at depth", depth)
                break

        return best


# Game tree from the given diagram
tree = [
    [ # N2: Minimizer
        [1, 4], # N4: Maximizer
        [7, 2] # N5: Maximizer
    ],
    [ # N3: Minimizer
        [3, 0], # N6: Maximizer
        [6, 5] # N7: Maximizer
    ]
]

result = alpha_beta(
    tree,
    0,
    -math.inf,
    math.inf,
    True
)
```

```
print("Optimal value at root N1:", result)
```